STEP 1: Load Dataset

The dataset was loaded using pandas and initial inspection was performed using head() and shape.

```
import pandas as pd

# load dataset
df = pd.read_excel("/content/Flight_dataset[4039].xlsx")

# check data
print(df.head())
print(df.shape)
```

```
   Unnamed: 0   airline   flight source_city departure_time stops  \
0           0  SpiceJet  SG-8709       Delhi        Evening  zero
1           1  SpiceJet  SG-8157       Delhi  Early_Morning  zero
2           2   AirAsia   I5-764       Delhi  Early_Morning  zero
3           3   Vistara   UK-995       Delhi        Morning  zero
4           4   Vistara   UK-963       Delhi        Morning  zero

    arrival_time destination_city    class  duration  days_left   price
0          Night           Mumbai  Economy      2.17        1.0  5953.0
1        Morning           Mumbai  Economy      2.33        1.0  5953.0
2  Early_Morning           Mumbai  Economy       NaN        1.0  5956.0
3      Afternoon           Mumbai  Economy      2.25        1.0  5955.0
4        Morning           Mumbai  Economy      2.33        1.0  5955.0
(300153, 12)
```

STEP 2: Data Understanding

The dataset contains flight booking details such as airline, source, destination, duration and price.

```
# check columns
print(df.columns)

# info
df.info()

# missing values
print(df.isnull().sum())
```

```
Index(['Unnamed: 0', 'airline', 'flight', 'source_city', 'departure_time',
       'stops', 'arrival_time', 'destination_city', 'class', 'duration',
       'days_left', 'price'],
      dtype='object')
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300153 entries, 0 to 300152
Data columns (total 12 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Unnamed: 0        300153 non-null  int64
 1   airline           300153 non-null  object
 2   flight            300153 non-null  object
 3   source_city       300153 non-null  object
 4   departure_time    300153 non-null  object
 5   stops             300153 non-null  object
 6   arrival_time      300153 non-null  object
 7   destination_city  300153 non-null  object
 8   class             300153 non-null  object
 9   duration          300141 non-null  float64
 10  days_left         300148 non-null  float64
 11  price             300148 non-null  float64
dtypes: float64(3), int64(1), object(8)
memory usage: 27.5+ MB
Unnamed: 0           0
airline              0
flight               0
source_city          0
departure_time       0
stops                0
```

```
arrival_time         0
destination_city     0
class                0
duration            12
days_left            5
price                5
dtype: int64
```

STEP 3: Data Cleaning

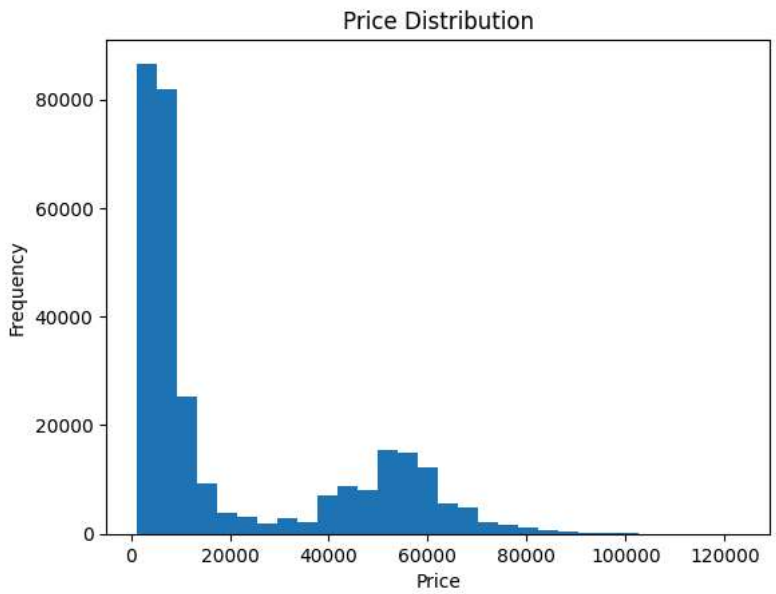
Missing values were removed to ensure a clean dataset for analysis.

```
# remove duplicates
df = df.drop_duplicates()

# fill missing values (if any)
df = df.dropna()
```

STEP 4: EDA

Ticket prices vary significantly across airlines

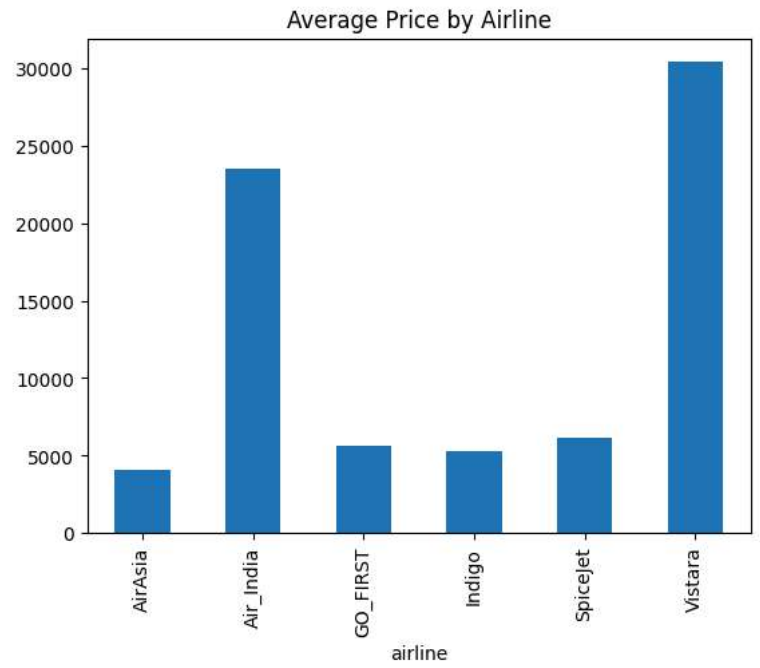
```
import matplotlib.pyplot as plt

plt.hist(df['price'], bins=30)
plt.title('Price Distribution')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```



Ticket prices are right-skewed, with most fares concentrated in lower ranges.

Some airlines show consistently higher pricing

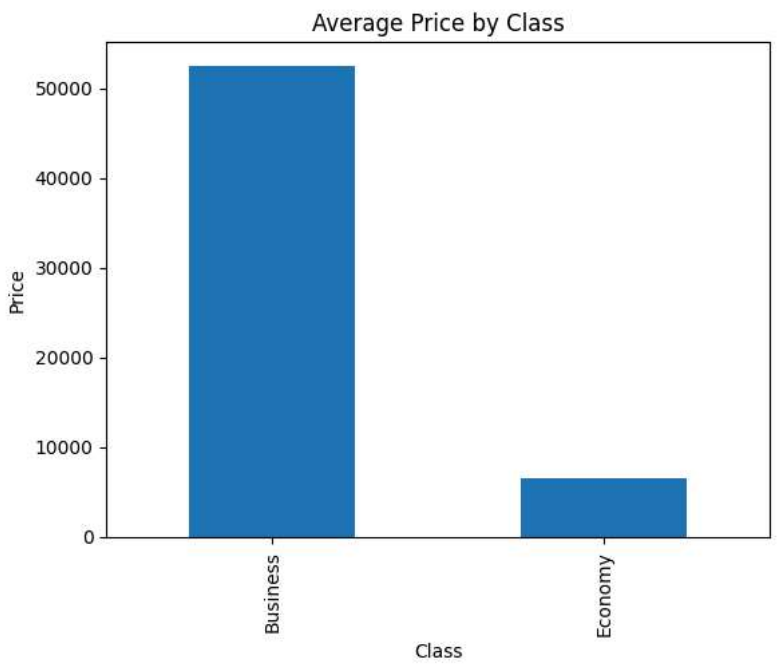
```
df.groupby('airline')['price'].mean().plot(kind='bar')
plt.title('Average Price by Airline')
plt.show()
```



Vistara and Air India show higher average fares, while AirAsia offers lower average prices.

Price by Class

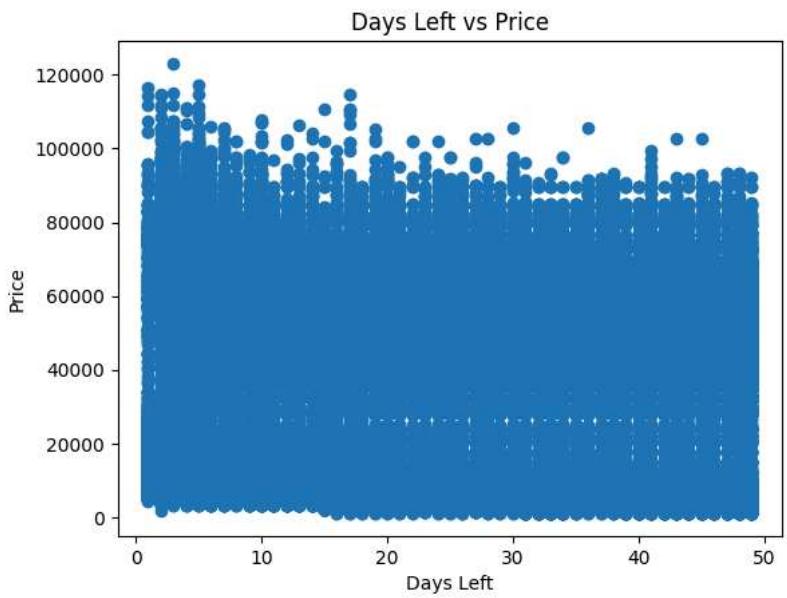
```
df.groupby('class')['price'].mean().plot(kind='bar')
plt.title("Average Price by Class")
plt.xlabel("Class")
plt.ylabel("Price")
plt.show()
```



Days Left vs Price

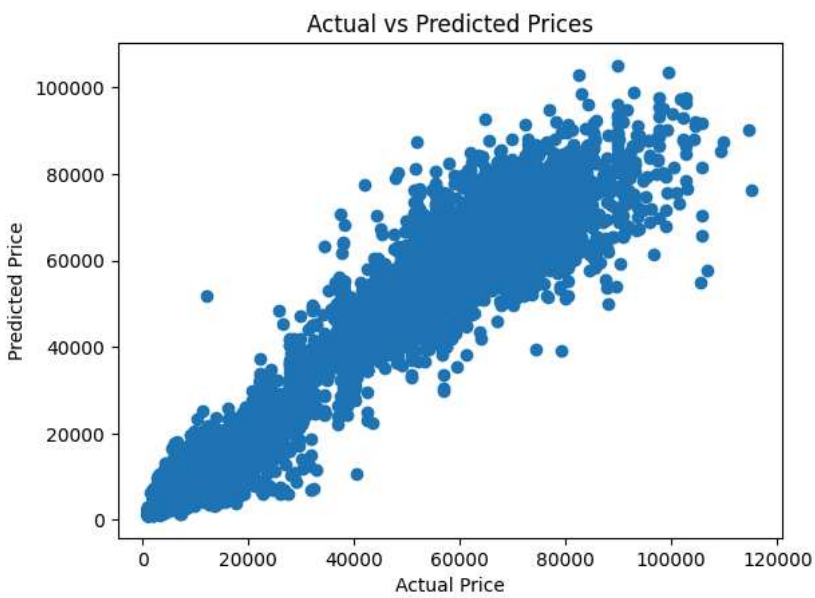
```
plt.scatter(df['days_left'], df['price'])
plt.title("Days Left vs Price")
plt.xlabel("Days Left")
```

```
plt.ylabel("Price")
plt.show()
```



Actual vs Predicted

```
plt.scatter(y_test, y_pred2)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Prices")
plt.show()
```



STEP 5: Hypothesis Testing

H0: Airline has no effect on ticket price.

H1: Airline significantly affects ticket price.

Since p-value = 0.0 (<0.05), H0 was rejected.

```
from scipy.stats import f_oneway
```

```
groups = [group['price'].values for name, group in df.groupby('airline')]
f_stat, p_value = f_oneway(*groups)

print(p_value)
```

```
0.0
```

ADDITIONAL HYPOTHESIS TESTS

1. Class vs Price

2. Stops vs Price

3. Source City vs Price

```
# Additional Hypothesis Tests

from scipy.stats import f_oneway

# 1. Class vs Price
groups_class = [group['price'].values for name, group in df.groupby('class')]
f_stat_class, p_value_class = f_oneway(*groups_class)

print("Class vs Price p-value:", p_value_class)

# 2. Stops vs Price
groups_stops = [group['price'].values for name, group in df.groupby('stops')]
f_stat_stops, p_value_stops = f_oneway(*groups_stops)

print("Stops vs Price p-value:", p_value_stops)

# 3. Source City vs Price
groups_source = [group['price'].values for name, group in df.groupby('source_city')]
f_stat_source, p_value_source = f_oneway(*groups_source)

print("Source City vs Price p-value:", p_value_source)
```

```
Class vs Price p-value: 0.0
Stops vs Price p-value: 0.0
Source City vs Price p-value: 4.663139452135189e-154
```

STEP 6: Feature Engineering

```
df = df.drop_duplicates()
df = df.dropna()

df = df.drop(['Unnamed: 0', 'flight'], axis=1, errors='ignore')

print(df.head())
print(df.shape)
```

```
    airline source_city departure_time stops arrival_time destination_city  \
0  SpiceJet       Delhi        Evening  zero        Night           Mumbai
1  SpiceJet       Delhi  Early_Morning  zero      Morning           Mumbai
3   Vistara       Delhi        Morning  zero    Afternoon           Mumbai
4   Vistara       Delhi        Morning  zero      Morning           Mumbai
5   Vistara       Delhi        Morning  zero    Afternoon           Mumbai

     class  duration  days_left   price
0  Economy      2.17        1.0  5953.0
1  Economy      2.33        1.0  5953.0
3  Economy      2.25        1.0  5955.0
4  Economy      2.33        1.0  5955.0
5  Economy      2.33        1.0  5955.0
(300133, 10)
```

STEP 7: Convert Categorical to Numeric

```
# convert text columns into numbers
df = pd.get_dummies(df, drop_first=True)

# check columns and shape
print(df.shape)
df.head()
```

```
(300133, 31)
```

	duration	days_left	price	airline_Air_India	airline_GO_FIRST	airline_Indigo	airline_SpiceJet	airline_Vistara	source_ci
0	2.17	1.0	5953.0	False	False	False	True	False	
1	2.33	1.0	5953.0	False	False	False	True	False	
3	2.25	1.0	5955.0	False	False	False	False	True	
4	2.33	1.0	5955.0	False	False	False	False	True	
5	2.33	1.0	5955.0	False	False	False	False	True	

5 rows × 31 columns

STEP 8: Define Features and Target

```
# independent variables
X = df.drop('price', axis=1)

# dependent variable
y = df['price']

print(X.shape)
print(y.shape)
```

```
(300133, 30)
(300133,)
```

STEP 9: Train-Test Split

The dataset was split into 80% training data and 20% testing data to evaluate model performance on unseen observations.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
```

```
(240106, 30)
(60027, 30)
```

STEP 10: Model 1 – Ridge Regression

Ridge Regression was selected as the first model to provide a strong baseline prediction method. It is an extension of Linear Regression that helps reduce overfitting and performs well when multiple features are present

```
from sklearn.linear_model import Ridge

# first regression model
model1 = Ridge()

model1.fit(X_train, y_train)

y_pred1 = model1.predict(X_test)
```

```
print("Model 1 completed")
```

```
Model 1 completed
```

STEP 11: Model 2 – Random Forest

Random Forest Regressor was selected as the second model because it can capture complex non-linear relationships between variables. It is useful for handling mixed data and often provides strong predictive performance.

```
from sklearn.ensemble import RandomForestRegressor

# second model
model2 = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

model2.fit(X_train, y_train)

y_pred2 = model2.predict(X_test)

print("Model 2 completed")
```

```
Model 2 completed
```

FEATURE IMPORTANCE

```
import pandas as pd

importance = pd.Series(model2.feature_importances_, index=X.columns)
importance.sort_values(ascending=False).head(10)
```

	0
class_Economy	0.880001
duration	0.058397
days_left	0.018739
airline_Air_India	0.005252
airline_Vistara	0.005077
destination_city_Delhi	0.003664
source_city_Delhi	0.003613
source_city_Mumbai	0.002446
destination_city_Mumbai	0.002214
destination_city_Hyderabad	0.001915

dtype: float64

OVERFITTING CHECK

```
print("Train R2:", model2.score(X_train, y_train))
print("Test R2:", model2.score(X_test, y_test))
```

```
Train R2: 0.9975036224517587
Test R2: 0.9851685752306415
```

CROSS VALIDATION

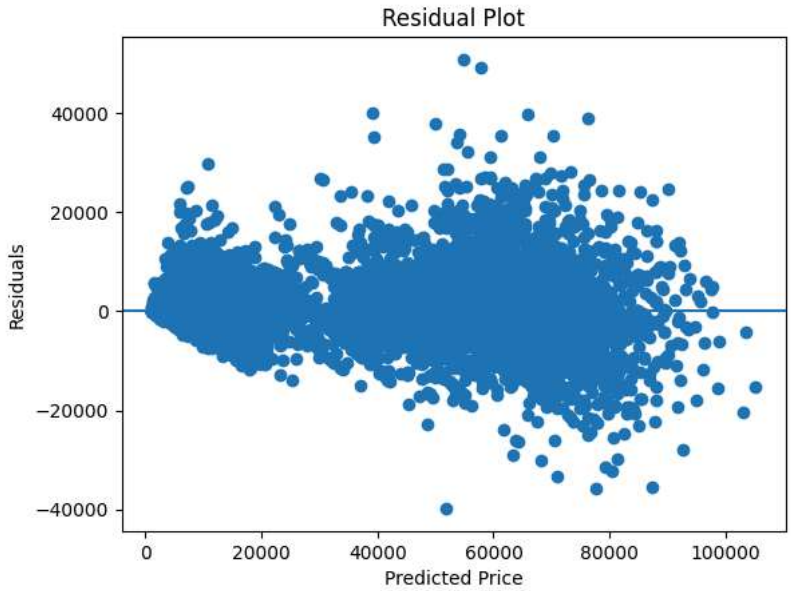
```
from sklearn.model_selection import cross_val_score
```

```
scores = cross_val_score(model2, X, y, cv=5, scoring='r2')
print("CV R2 Mean:", scores.mean())
```

```
CV R2 Mean: 0.5676099966235617
```

RESIDUAL PLOT

```
residuals = y_test - y_pred2

plt.scatter(y_pred2, residuals)
plt.axhline(0)
plt.title("Residual Plot")
plt.xlabel("Predicted Price")
plt.ylabel("Residuals")
plt.show()
```



STEP 12: Evaluate Both Models

Both regression models were evaluated using R^2 Score and RMSE. Random Forest outperformed Ridge Regression by achieving higher predictive performance and lower error. R^2 measures explained variance, while RMSE measures prediction error. Therefore, Random Forest was selected as the final model.

```
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np

def check_result(name, y_true, y_pred):
    r2 = r2_score(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))

    print(name)
    print("R2 Score :", round(r2,4))
    print("RMSE     :", round(rmse,2))
    print("--------------------")

check_result("Ridge Regression", y_test, y_pred1)
check_result("Random Forest", y_test, y_pred2)
```

```
Ridge Regression
R2 Score : 0.9112
RMSE     : 6764.46
--------------------
Random Forest
R2 Score : 0.9852
```

```
RMSE     : 2764.87
--------------------
```

Evaluation Metrics RMSLE & MAPE

```
from sklearn.metrics import mean_absolute_percentage_error
import numpy as np

def extra_metrics(y_true, y_pred):
    # Fix negative values issue
    y_pred_safe = np.maximum(y_pred, 0)
    y_true_safe = np.maximum(y_true, 0)

    rmsle = np.sqrt(np.mean((np.log1p(y_pred_safe) - np.log1p(y_true_safe))**2))
    mape = mean_absolute_percentage_error(y_true, y_pred)

    print("RMSLE:", round(rmsle,4))
    print("MAPE :", round(mape,4))

print("Ridge Extra Metrics")
extra_metrics(y_test, y_pred1)

print("\nRandom Forest Extra Metrics")
extra_metrics(y_test, y_pred2)
```

```
Ridge Extra Metrics
RMSLE: 1.9811
MAPE : 0.4616

Random Forest Extra Metrics
RMSLE: 0.1385
MAPE : 0.0708
```

STEP 13: Final Conclusion

Two regression models were developed to predict flight prices. Ridge Regression provided strong baseline performance, while Random Forest delivered the best results with an R^2 score of 0.9847 and lower RMSE. This indicates that flight pricing is influenced by complex non-linear factors such as airline, class, route and booking timing.